

# Documentação de Projeto: EpicForge - O Compêndio de RPG

**Status:** Sprint 1 (Entrega 15/05)

## 1. Definição do Escopo

O EpicForge é uma plataforma web voltada para entusiastas de RPG de mesa (focado em D&D 5e), permitindo que usuários consultem um compêndio completo de itens, magias e classes, além de criarem, gerenciarem e compartilhem builds de personagens (fichas simplificadas). O sistema resolve a fragmentação de informações durante as sessões de jogo, centralizando dados oficiais e permitindo a persistência de customizações dos jogadores.

## 2. Justificativa da Stack Tecnológica

| Tecnologia                | Papel          | Justificativa                                                                                                                                 |
|---------------------------|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| TypeScript                | Linguagem      | Garante tipagem estática, reduzindo erros em tempo de execução e facilitando a manutenção e a criação de testes automatizados exigidos.       |
| React.js                  | Front-end      | Arquitetura baseada em componentes que facilita a criação de interfaces dinâmicas (como o construtor de builds) e reaproveitamento de código. |
| Node.js + Fastify/Express | Back-end       | Alta performance em operações I/O e ecossistema robusto para integração com APIs externas e ferramentas de teste.                             |
| PostgreSQL                | Banco de Dados | Banco relacional que garante integridade referencial entre usuários, personagens e itens, essencial para um sistema de regras complexo.       |
| Prisma ORM                | Integração BD  | Facilita a modelagem e migração de dados com total suporte a TypeScript.                                                                      |

| Tecnologia        | Papel  | Justificativa                                                                                              |
|-------------------|--------|------------------------------------------------------------------------------------------------------------|
| Jest / Playwright | Testes | Ferramentas líderes de mercado para cobrir os três níveis de testes exigidos (unitário, integração e E2E). |

### 3. Documentação de Casos de Uso (Mínimo 15)

Abaixo, os fluxos principais da aplicação seguindo o padrão exigido:

- **UC01: Cadastro de Usuário** (Ator: Jogador | Fluxo: Criar conta com email/senha).
- **UC02: Login de Usuário** (Ator: Jogador | Fluxo: Autenticação e geração de Token JWT).
- **UC03: Consulta de Itens** (Ator: Jogador | Fluxo: Buscar itens no compêndio via API externa).
- **UC04: Filtro Avançado de Itens** (Ator: Jogador | Fluxo: Filtrar por raridade ou tipo de dano).
- **UC05: Visualização de Detalhes de Classe** (Ator: Jogador | Fluxo: Ver proficiências e habilidades de uma classe).
- **UC06: Criação de Nova Build** (Ator: Jogador | Fluxo: Iniciar uma nova ficha de personagem vazia).
- **UC07: Edição de Nome e Descrição da Build** (Ator: Jogador | Fluxo: Atualizar metadados do personagem).
- **UC08: Adição de Item ao Inventário da Build** (Ator: Jogador | Fluxo: Persistir relação entre build e item).
- **UC09: Remoção de Item da Build** (Ator: Jogador | Fluxo: Deletar item específico da ficha).
- **UC10: Seleção de Classe para Build** (Ator: Jogador | Fluxo: Definir a classe base do personagem).
- **UC11: Cálculo Automático de Atributos** (Ator: Sistema | Fluxo: Somar bônus de itens ao status base).
- **UC12: Listagem de Builds do Usuário** (Ator: Jogador | Fluxo: Visualizar todos os personagens criados).
- **UC13: Exclusão de Build** (Ator: Jogador | Fluxo: Deletar permanentemente uma build do banco).
- **UC14: Favoritar Build Pública** (Ator: Jogador | Fluxo: Salvar build de outro usuário como referência).
- **UC15: Sincronização de Dados com API Externa** (Ator: Sistema | Fluxo: Atualizar cache local com dados da D&D 5e API).

## 4. Modelagem do Banco de Dados (Entidade-Relacionamento)

Estrutura lógica proposta para o PostgreSQL:

- Table: users (id, name, email, password\_hash, created\_at)
- Table: characters (id, name, level, user\_id [FK], class\_id, race\_id)
- Table: builds\_items (id, character\_id [FK], item\_external\_index, quantity)
- Table: favorites (id, user\_id [FK], character\_id [FK])

## 5. Integração com API Externa

**API Escolhida:** D&D 5e API (<https://www.dnd5eapi.co/>)

**Justificativa:** API pública, gratuita e extremamente bem documentada que fornece todos os dados necessários (monstros, magias, itens e classes) em formato JSON, eliminando a necessidade de cadastro manual de milhares de entidades e permitindo focar na lógica de construção de builds.